

FICHE DE RÉVISION ULTIME

QCM JavaScript / Node.js / POO ES6

Basée sur le cours de Zineb CHERRADI - ESISA

1 Types et Valeurs en JavaScript

1.1 Type Number

JavaScript possède un **unique type numérique**. Il ne fait **pas** la distinction entre les entiers (`int`) et les flottants (`double`).

```
1 typeof 10    // "number"
2 typeof 10.5  // "number"
```

1.2 Le piège de `typeof null`

Attention / Piège

`typeof null` retourne `"object"`. C'est un bug historique de JavaScript, mais c'est un classique des QCM !

1.3 `null` vs `undefined`

- **`null`** : Représente l'absence *intentionnelle* de valeur d'un objet.
- **`undefined`** : Variable déclarée mais non initialisée, ou propriété inexistante.

1.4 Hoisting (Remontée) avec `var`

Avec `var`, la déclaration est remontée en haut du scope, mais pas l'initialisation.

```
1 console.log(x); // Affiche "undefined" (PAS d'erreur !)
```

```
2 var x = 10;
```

2 Contexte Global

Environnement	Objet Global
Navigateur (Browser)	<code>window</code>
Node.js	<code>global</code>

À Retenir

Dans un navigateur, une variable globale `var x = 10;` devient automatiquement une propriété de `window` (accessible via `window.x`).

3 Valeurs Truthy et Falsy

Attention / Piège

Seules **6 valeurs** sont considérées comme **falsy** (fausses) dans un test conditionnel. Tout le reste est **truthy** (vrai).

Les 6 valeurs Falsy : false, 0, "" (chaîne vide), null, undefined, NaN.

Exemples de valeurs Truthy (pièges courants) :

```
1 if ("0") { ... } // TRUE (c'est une chaîne non vide)
2 if ([]) { ... } // TRUE (tableau vide)
3 if ({}) { ... } // TRUE (objet vide)
```

4 Comparaisons

— **== (Égalité large)** : Effectue une *conversion de type* (coercition).

5 == "5" → true

— **=== (Égalité stricte)** : Pas de conversion de type.

5 === "5" → false

5 Modules et Architecture Node.js

5.1 Systèmes de Modules

— **CommonJS** : Utilise `require()` et `module.exports` / `exports`. (Standard Node.js).

— **ES6 Modules** : Utilise `import` et `export`.

À Retenir

`exports` est le paramètre de la "wrapper function" qui permet de publier les objets d'un module Node.js.

5.2 Répertoires et Architecture en Couches

Dossier / Couche	Rôle
node_modules	Contient les dépendances npm (avec un underscore _).
config	Configuration de la BDD (ex : <code>mysql.config.js</code>).
repositories	Accès aux données (requêtes SQL, lecture JSON).
services	Logique métier (traitement des données).
controllers	Gestion des routes et requêtes HTTP.

6 Objets, Prototypes et POO ES6

6.1 Objets et Prototypes

— `Object.keys(x)` : Retourne les propriétés (clés) d'un objet `x`.

— **Prototype** : Mécanisme qui permet à plusieurs objets de *partager* des méthodes.

— **Chaîne de prototypes** : Si une propriété n'est pas trouvée dans un objet, JS cherche dans son prototype, puis dans le prototype du prototype, etc. (Retourne `undefined` si non trouvé, *pas d'erreur*).

6.2 Classes ES6 (POO)

```
1 class Point {  
2   #id = 100;           // Propriété PRIVÉE (préfixe #)  
3   static MAX_X = 1000; // Propriété STATIQUE (à la classe)  
4  
5   constructor(x, y) { this.x = x; this.y = y; }  
6  
7   get id() { return this.#id; } // Getter  
8   set id(val) { this.#id = val; } // Setter  
9  
10  static info() { console.log(Point.MAX_X); }  
11 }
```

Attention / Piège

Membres Statiques (static) : Ils appartiennent à la classe, pas aux instances. On y accède via `Point.MAX_X`. Une méthode statique ne peut pas utiliser `this` pour accéder aux propriétés d'instance (`this.x` sera `undefined`).

6.3 Héritage

```
1 class City extends Point {  
2   constructor(name, x, y) {  
3     super(x, y); // OBLIGATOIRE : appel au constructeur parent  
4     this.name = name;  
5   }  
6 }
```

7 Programmation Asynchrone

- **Promises :** Gérées avec `.then()` (succès) et `.catch()` (erreur).
- **Async/Await :** Syntaxe moderne pour rendre le code asynchrone plus lisible (ressemble à du synchrone).

À Retenir

Règles Async/Await :

1. `await` ne peut être utilisé que dans une fonction déclarée `async`.
2. Une fonction `async` retourne toujours une `Promise`.
3. `await` bloque l'exécution à l'intérieur de la fonction, mais pas tout le programme.

8 Récapitulatif Express pour le QCM

Question / Concept	Réponse / Syntaxe
Type de <code>null</code>	<code>"object"</code>
Contexte global (Navigateur)	<code>window</code>
Contexte global (Node.js)	<code>global</code>
Valeur falsy parmi <code>"0"</code> , <code>[]</code> , <code>0</code> , <code>{}</code>	<code>0</code>
Résultat de <code>5 == "5"</code>	<code>true</code>
Méthode pour les clés d'un objet	<code>Object.keys(x)</code>
Partage de méthodes entre objets	Prototype
Propriété non trouvée en lecture	Cherche dans la chaîne de prototypes
Avantage de <code>async/await</code>	Syntaxe plus lisible
Publier un objet de module Node.js	<code>exports</code>
Dossier des dépendances Node.js	<code>node_modules</code>
Système de modules de <code>require()</code>	CommonJS
Couche contenant la logique métier	services
Où placer la config MySQL ?	config

Bon courage pour ton examen !